Practical Kubernetes Administration and Troubleshooting

Day 4: Security, Monitoring, and Troubleshooting

Instructor: Chad M. Crowell

Day 3 Knowledge Check

8 quick questions before we dive into Day 4

Quiz: Day 3 Recap

1. What DNS name format does Kubernetes automatically assign to every Service?
2. True or False: A pod with no NetworkPolicy selecting it is fully open to traffic from all other pods.
3. What's the difference between a taint's `NoSchedule` effect and its `NoExecute` effect?
4. Which Job field controls how many pods run **at the same time**?
5. What object does Kubernetes automatically create to track which pod IPs sit behind a Service, and which `kubectl` command inspects it when traffic isn't routing?
6. True or False: A StorageClass with `volumeBindingMode: WaitForFirstConsumer` binds the PVC to a PV immediately, before the pod is scheduled.
7. Which PV reclaim policy should you use for production databases to avoid accidental data loss when a PVC is deleted?

Quiz: Answers

1. `<service-name>.<namespace>.svc.cluster.local`
2. **True** – with no policy selecting it, a pod allows all ingress/egress traffic by default
3. `NoSchedule` blocks new non-tolerating pods from scheduling but leaves existing pods alone; `NoExecute` does that **and evicts** already-running pods that don't tolerate it
4. `parallelism` – `completions` controls the total number of successful runs needed, not concurrency
5. **The** `Endpoints` **object** – `kubectl get endpoints <svc-name>` ; an empty list means the Service's selector doesn't match any pod labels
6. **False** – `WaitForFirstConsumer` **delays** binding until a pod using the PVC is scheduled, which avoids provisioning storage in the wrong zone
7. `Retain` – the PV and underlying volume survive PVC deletion instead of being deleted automatically

Day 3 Recap

What We Covered Yesterday

- Kubernetes Networking Model – flat network, pod IPs, CNI
- Service Types – ClusterIP, NodePort, LoadBalancer, ExternalName
- DNS and CoreDNS – service discovery and name resolution
- Network Policies – restricting pod-to-pod traffic
- Scheduling – taints, tolerations, node affinity, DaemonSets
- Jobs and CronJobs – batch and scheduled workloads
- Persistent Storage – PVs, PVCs, StorageClasses, Linode CSI
- Ingress and Gateway API – routing external HTTP traffic

Where We Left Off

- A running cluster with services, storage, and ingress configured
- Ready to lock it down, monitor it, and keep it healthy

Day 4 Agenda

Time	Topic
Morning	Security Essentials
	RBAC – Roles, ClusterRoles, Bindings
	Service Accounts and Security Contexts
	Pod Security Standards
Midday	Monitoring and Observability
	Helm and Package Management
	Prometheus, Grafana, and Alertmanager
Afternoon	Troubleshooting and Cluster Operations
	Diagnosing Pods, Nodes, and Networking

Labs Today

- RBAC and access control exercises
- Multi-node cluster operations and validation
- Monitoring setup with Prometheus and Grafana
- Debug simulated cluster issues
- Cluster upgrade – control plane and worker nodes
- etcd backup and restore

Security Essentials

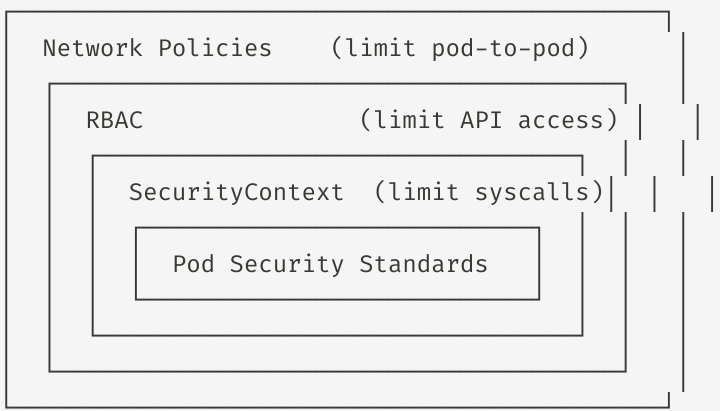
RBAC, Service Accounts, and Security Contexts

Why Kubernetes Security Matters

Kubernetes clusters are high-value targets. A misconfigured cluster can expose:

- **All secrets** in the cluster (API keys, passwords, TLS certs)
- **The underlying node filesystem** via hostPath volumes
- **The cloud provider API** via the node's IAM role
- **Other tenants' workloads** if namespaces are not isolated

The Defense-in-Depth Model



Role-Based Access Control (RBAC)

RBAC controls **who can do what** in the cluster.

The Four RBAC Objects

Object	Scope	Purpose
`Role`	Namespace	Defines permissions within a namespace
`ClusterRole`	Cluster-wide	Defines permissions across all namespaces or non-namespaced resources

The Subject Types

```
Subject
├── User        (human, identified by cert CN)
├── Group       (identified by cert O field)
└── ServiceAccount  (pod identity)
```

The Permission Model

```
Subject  +  Role  =  RoleBinding

jane     +  pod-reader  =  can list pods
                           in namespace "default"
```

Roles and ClusterRoles

A **Role** grants permissions within a single namespace. A **ClusterRole** grants permissions cluster-wide or on non-namespaced resources (Nodes, PVs, etc.).

```yaml
# Role — namespaced
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]          # "" = core API group
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
```

```yaml
# ClusterRole — cluster-wide
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["metrics.k8s.io"]
  resources: ["nodes", "pods"]
  verbs: ["get", "list"]
```

RoleBindings and ClusterRoleBindings

```yaml
# RoleBinding — grant Role to a user in a namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: jane               # must match CN in client cert
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

```bash
# check what permissions a user has
kubectl auth can-i list pods --as=jane
kubectl auth can-i list pods --as=jane -n production
kubectl auth can-i "*" "*"    # check if you're cluster-admin
```

A **RoleBinding can reference a ClusterRole** — this is a common pattern to define permissions once as a ClusterRole, then grant them per-namespace via RoleBindings.

Built-In ClusterRoles

Kubernetes ships with several pre-built ClusterRoles:

ClusterRole	What It Grants
`cluster-admin`	Full access to everything — use sparingly
`admin`	Full namespace access (all resources except ResourceQuota)
`edit`	Read/write most namespace resources; no RBAC management
`view`	Read-only access to most namespace resources

```
# list all ClusterRoles
kubectl get clusterroles

# list cluster-admin
kubectl get clusterrole cluster-admin

# get client-certificate-data
kubectl config view --raw

# base64 decode
echo "" | base64 -d | openssl x509 -text -noout | grep -A1

# cluster role binding
kubectl get clusterrolebinding cluster-admin

# inspect the built-in view role
kubectl describe clusterrole view

# grant the edit ClusterRole to a user in a specific names
kubectl create rolebinding jane-edit \
  --clusterrole=edit \
  --user=jane \
  --namespace=staging
```

RBAC for Groups

Groups let you grant permissions to multiple users at once. The `O` field in the x509 certificate becomes the group name.

```
# create keys for bob
openssl genrsa -out bob.key 2048

# create a user in the "dev-team" group
openssl req -new -key bob.key -out bob.csr \
  -subj "/CN=bob/O=dev-team"

# sign the cert with the cluster CA
sudo openssl x509 -req -in bob.csr \
  -CA /etc/kubernetes/pki/ca.crt \
  -CAkey /etc/kubernetes/pki/ca.key \
  -CAcreateserial -out bob.crt -days 365
```

```
# grant the dev-team group edit access to the staging name
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-team-edit
  namespace: dev
subjects:
- kind: Group
  name: dev-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: edit
  apiGroup: rbac.authorization.k8s.io
```

Service Accounts

A **ServiceAccount** gives pods an identity so they can authenticate to the Kubernetes API.

Every Pod Gets One

By default, pods use the `default` ServiceAccount in their namespace. It has minimal permissions.

```
# list service accounts
kubectl get serviceaccounts

# inspect the token mounted in a pod
kubectl exec <pod> -- \
  cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

Create a Dedicated ServiceAccount

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: monitoring-agent
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: monitoring-agent-binding
subjects:
- kind: ServiceAccount
  name: monitoring-agent
  namespace: monitoring
roleRef:
  kind: ClusterRole
  name: view
  apiGroup: rbac.authorization.k8s.io
```

Using a ServiceAccount in a Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: api-client
spec:
  serviceAccountName: monitoring-agent   # use this SA instead of default
  automountServiceAccountToken: true     # default true; set false if pod doesn't need API access
  containers:
  - name: client
    image: curlimages/curl
    command: ["sh", "-c", "sleep 3600"]
```

```bash
# from inside the pod, call the API server using the mounted token
kubectl exec -it api-client -- sh

TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://kubernetes.default.svc.cluster.local/api/v1/namespaces/default/pods
```

In Kubernetes 1.22+, ServiceAccount tokens are **time-limited and audience-bound** (projected tokens). They expire after 1 hour by default. The kubelet automatically rotates them.

SecurityContext

A **SecurityContext** defines privilege and access control settings for a pod or container.

Pod-level SecurityContext

Applies to all containers in the pod:

```yaml
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000        # volume ownership
    seccompProfile:
      type: RuntimeDefault
```

Container-level SecurityContext

Overrides pod-level settings per container:

```yaml
  containers:
  - name: app
    image: myapp:latest
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
        add: ["NET_BIND_SERVICE"]
```

Principle of least privilege: Drop all Linux capabilities and only add back what the app needs. `readOnlyRootFilesystem: true` prevents attackers from writing to the container filesystem.

Pod Security Standards

Kubernetes 1.25+ enforces security via **Pod Security Admission** — a built-in admission controller that checks pods against one of three security profiles:

Profile	Restrictions	Use Case
`privileged`	No restrictions	System components (CNI, CSI)
`baseline`	Blocks known escalation paths	Most applications
`restricted`	Full hardening (non-root, no privesc, seccomp)	Security-sensitive workloads

Pod Security Standards (cont.)

```
# label a namespace to enforce the restricted profile
kubectl label namespace production \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
```

```
# test what would happen without enforcing
kubectl label namespace staging \
  pod-security.kubernetes.io/warn=baseline
```

`enforce` = reject violating pods. `warn` = allow but emit a warning. `audit` = allow and log to audit log.

```
# add PodSecurity to kube-apiserver.yaml
sudo sed -i 's/--enable-admission-plugins=\([^"]*\)/--enable-admission-plugins=\1,PodSecurity/' /etc/kubernetes/manifest
```

RBAC Troubleshooting

```
# check if a user/SA can perform an action
kubectl auth can-i create deployments --as=jane
kubectl auth can-i create deployments --as=jane -n production
kubectl auth can-i "*" "*" --as=system:serviceaccount:default:my-sa

# see all permissions for a service account
kubectl auth can-i --list --as=system:serviceaccount:default:monitoring-agent

# describe who can do what
kubectl get rolebindings,clusterrolebindings -A | grep jane

# common error: pods get Forbidden when calling API
# check the pod's service account and its bindings
kubectl get pod <name> -o jsonpath='{.spec.serviceAccountName}'
kubectl get rolebinding,clusterrolebinding -A -o yaml | grep <sa-name>
```

RBAC Troubleshooting (cont.)

Common RBAC Mistakes

Mistake	Symptom	Fix
Wrong namespace on RoleBinding	Works in default, fails in staging	Check `metadata.namespace` on the binding
Binding a Role to wrong subject type	Forbidden even with correct name	Check `subjects[].kind` (User vs ServiceAccount)
Missing API group	Verb allowed but still Forbidden	Add the correct `apiGroups` entry to the Role

Service Accounts and Security Contexts

Pod identity and runtime security

Securing Workloads: Best Practices

What to Always Do

- Run as non-root (`runAsNonRoot: true`)
- Drop all capabilities (`drop: ["ALL"]`)
- Set `readOnlyRootFilesystem: true`
- Use dedicated ServiceAccounts per workload
- Set `automountServiceAccountToken: false` when the pod doesn't need API access
- Apply resource requests and limits

What to Never Do

- Run containers as root (UID 0)
- Use `privileged: true` for app containers
- Mount `/var/run/docker.sock` in a container
- Use `hostPID: true` or `hostNetwork: true` without good reason
- Give ServiceAccounts `cluster-admin`
- Store secrets in environment variables when a secret store is available

Secrets Security

Kubernetes Secrets are base64-encoded, **not encrypted**, in etcd by default.

Encryption at Rest

Enable encryption for secrets in etcd with an `EncryptionConfiguration`:

```yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources:
  - secrets
  providers:
  - aescbc:
      keys:
      - name: key1
        secret: <base64-encoded 32-byte key>
  - identity: {}    # fallback for unencrypted secrets
```

Secrets Security (cont.)

```
# reference the config in kube-apiserver (add to static pod manifest)
--encryption-provider-config=/etc/kubernetes/encryption-config.yaml

# verify a secret is stored encrypted in etcd
sudo etcdctl get /registry/secrets/default/my-secret \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key | hexdump -C | head
```

In production, use an **External Secrets Operator** with HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault to manage secrets outside the cluster.

Monitoring and Observability

Prometheus, Grafana, and Alertmanager

The Three Pillars of Observability

Pillar	What It Tells You	Tool
Metrics	Numeric measurements over time (CPU, memory, request rate)	Prometheus
Logs	Discrete events with context	Loki, Elasticsearch
Traces	Request paths across distributed services	Jaeger, Tempo

Why All Three?

- **Metrics** tell you *something is wrong* (CPU spike)
- **Logs** tell you *what happened* (error message at that time)
- **Traces** tell you *where the time went* (which service was slow)

> Today we focus on metrics with Prometheus/Grafana — the most commonly deployed observability stack in Kubernetes.

Introduction to Helm

Helm is the package manager for Kubernetes. It bundles YAML manifests into **charts** that can be versioned, shared, and configured.

Key Concepts

Term	Meaning
Chart	A package of Kubernetes manifests
Release	An installed instance of a chart
Repository	A collection of charts (like apt/npm)
Values	Configuration overrides for a chart

Install Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scri

helm version
```

Add a Repository

```
helm repo add stable https://charts.helm.sh/stable
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update
```

Helm Basics

```
# search for charts
helm search repo prometheus

# inspect a chart before installing
helm show values prometheus-community/kube-prometheus-stack

# install a chart
helm install my-release prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace

# list installed releases
helm list -A

# upgrade a release with new values
helm upgrade my-release prometheus-community/kube-prometheus-stack \
  --set grafana.adminPassword=newpassword

# uninstall a release
helm uninstall my-release -n monitoring

# package and template (dry run)
helm template my-release prometheus-community/kube-prometheus-stack \
  --values my-values.yaml | head -50
```

kube-prometheus-stack

The **kube-prometheus-stack** Helm chart installs the full monitoring stack in one command:

```
kube-prometheus-stack
├── Prometheus Operator     manages Prometheus instances via CRDs
├── Prometheus              scrapes metrics from targets
├── Alertmanager            routes alerts to Slack, PagerDuty, email
├── Grafana                 dashboards and visualization
├── kube-state-metrics      exposes cluster-level metrics
├── node-exporter           exposes node-level metrics
└── Prometheus rules        pre-built alerting rules
```

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --set grafana.adminPassword=admin123 \
  --set prometheus.prometheusSpec.retention=7d
```

```
# verify everything is running
kubectl get pods -n monitoring
kubectl get svc -n monitoring
```

Prometheus

Prometheus is a time-series metrics database. It **scrapes** HTTP endpoints that expose metrics in a standard format.

How Prometheus Discovers Targets

```
ServiceMonitor (CRD) → tells Prometheus which Services to scrape
PodMonitor (CRD)     → tells Prometheus which Pods to scrape
```

```
# port-forward to access Prometheus UI
kubectl port-forward svc/monitoring-kube-prometheus-prometheus \
  9090:9090 -n monitoring
# open http://localhost:9090

# create SSH tunnel (local port forward)
ssh -f -N \
  -L 3000:localhost:3000 \
  -L 9090:localhost:9090 \
  admin@controlplane

# OPTIONAL reverse tunnel (remote port forward)
ssh -R 9090:localhost:9090 admin@controlplane

# find the service to kill the tunnel
ps aux | grep "prom-tunnel\|port-forward"
```

Prometheus (cont.)

Basic PromQL Queries

```
# CPU usage per pod
rate(container_cpu_usage_seconds_total{namespace="default"}[5m])

# Memory usage per pod
container_memory_working_set_bytes{namespace="default"}

# Pod restart count
kube_pod_container_status_restarts_total{namespace="default"}

# Number of ready pods per deployment
kube_deployment_status_replicas_ready
```

Grafana

Grafana visualizes Prometheus metrics as dashboards.

```
# port-forward to access Grafana UI
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
# open http://localhost:3000
# default credentials: admin / admin123

# create SSH tunnel
ssh -f -N \
  -L 3000:localhost:3000 \
  -L 9090:localhost:9000 \
  admin@controlplane
```

Grafana (cont.)

Pre-Built Dashboards

The kube-prometheus-stack includes production-ready dashboards:

Dashboard	What It Shows
Kubernetes / Cluster	Node CPU, memory, pod count
Kubernetes / Nodes	Per-node resource usage
Kubernetes / Pods	Per-pod CPU, memory, restarts
Kubernetes / Deployments	Replica status, rollout history
Node Exporter Full	Detailed node metrics (disk, network, load)

Grafana (cont.)

```
# import additional community dashboards
# go to Dashboards → Import → enter dashboard ID from grafana.com/dashboards
# popular IDs: 6417 (k8s cluster), 1860 (node exporter)
```

kubectl top

Before Prometheus is set up, `kubectl top` gives quick resource usage from the **Metrics Server**:

```
# install metrics-server (if not already installed)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# node resource usage
kubectl top nodes
# NAME       CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
# cp         312m         15%    1823Mi          47%
# worker-1   188m         9%     1456Mi          37%

# pod resource usage
kubectl top pods -A --sort-by=memory
kubectl top pods -n default
```

> `kubectl top` pulls from the Metrics Server API (short-term, in-memory). Prometheus retains historical data for trend analysis and alerting.

Alertmanager

Alertmanager receives alerts from Prometheus and routes them to notification channels.

```yaml
# configure Alertmanager via a Secret (or Helm values)
apiVersion: v1
kind: Secret
metadata:
  name: alertmanager-monitoring-kube-prometheus-alertmanager
  namespace: monitoring
stringData:
  alertmanager.yaml: |
    global:
      slack_api_url: 'https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK'
    route:
      receiver: 'slack'
      group_wait: 30s
      group_interval: 5m
      repeat_interval: 4h
    receivers:
    - name: 'slack'
      slack_configs:
      - channel: '#alerts'
        title: '{{ .GroupLabels.alertname }}'
        text: '{{ range .Alerts }}{{ .Annotations.summary }}{{ end }}'
```

Alertmanager (cont.)

```
kubectl port-forward svc/monitoring-kube-prometheus-alertmanager \
  9093:9093 -n monitoring
```

Creating a PrometheusRule

Define custom alerting rules with the `PrometheusRule` CRD:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: pod-alerts
  namespace: monitoring
  labels:
    release: monitoring    # must match the Prometheus Operator's ruleSelector
spec:
  groups:
  - name: pod.rules
    rules:
    - alert: PodCrashLooping
      expr: rate(kube_pod_container_status_restarts_total[15m]) > 0
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Pod {{ $labels.pod }} is crash looping"
        description: "Pod {{ $labels.pod }} in namespace {{ $labels.namespace }} has restarted more than once in 15 minu
    - alert: PodNotReady
      expr: kube_pod_status_ready{condition="true"} == 0
      for: 10m
      labels:
```

Troubleshooting and Cluster Operations

Diagnosing issues, upgrading clusters, and managing etcd

Systematic Troubleshooting

A structured approach prevents wasted time. Always work **outside-in**:

```
1. Is the cluster healthy?
   kubectl get nodes
   kubectl get pods -n kube-system

2. What is the resource's current state?
   kubectl get <resource> -o wide

3. What does the full spec and status say?
   kubectl describe <resource>

4. What are the logs saying?
   kubectl logs <pod> [--previous]

5. What events are in the cluster?
   kubectl get events --sort-by=.metadata.creationTimestamp -A

6. Can I exec into the pod and test from inside?
   kubectl exec -it <pod> -- sh
```

Diagnosing Pod Issues

Pod States Reference

State	Meaning
`Pending`	Waiting to be scheduled
`ContainerCreating`	Image pulling or volumes mounting
`Running`	At least one container running
`CrashLoopBackOff`	Exiting repeatedly
`OOMKilled`	Exceeded memory limit
`ImagePullBackOff`	Can’t pull the image
`Terminating`	Being deleted (stuck = finalizer)
`Evicted`	Removed due to node resource pressure

Diagnostic Commands

```
# full pod details and events
kubectl describe pod <name>

# current logs
kubectl logs <name>

# logs from previous container run
kubectl logs <name> --previous

# stream logs
kubectl logs -f <name>

# exec into a running container
kubectl exec -it <name> -- sh

# copy files to/from a pod
kubectl cp <pod>:/path/to/file ./local-file
```

Diagnosing Node Issues

```
# check node status and conditions
kubectl get nodes -o wide
kubectl describe node <node-name>
```

Node Conditions to Watch

Condition	Meaning
`Ready=True`	Node is healthy
`MemoryPressure=True`	Node is low on memory
`DiskPressure=True`	Node is low on disk
`PIDPressure=True`	Too many processes on node
`NetworkUnavailable=True`	CNI plugin not configured

Common Networking Failures

Pod Can’t Reach Service

```
# 1. does the service exist?
kubectl get svc <name>

# 2. does it have endpoints?
kubectl get endpoints <name>
# empty = selector mismatch

# 3. do the labels match?
kubectl describe svc <name> | grep Selector
kubectl get pods --show-labels

# 4. test from inside a pod
kubectl run debug --image=busybox --rm -it \
  --restart=Never -- sh
wget -qO- http://<svc-name>.<ns>.svc.cluster.local
```

DNS Not Resolving

```
# 1. is CoreDNS running?
kubectl get pods -n kube-system \
  -l k8s-app=kube-dns

# 2. test DNS from a pod
kubectl run dns-test --image=busybox \
  --rm -it --restart=Never -- \
  nslookup kubernetes.default

# 3. check CoreDNS logs
kubectl logs -n kube-system \
  -l k8s-app=kube-dns

# 4. check pod resolv.conf
kubectl exec <pod> -- \
  cat /etc/resolv.conf
```

Common Deployment Failures

```
# deployment not rolling out
kubectl rollout status deployment/<name>
kubectl describe deployment <name>

# ReplicaSet not creating pods
kubectl get rs
kubectl describe rs <name>

# pods created but all Pending
kubectl get pods -o wide
kubectl describe pod <name>
# common: "Insufficient cpu",
# "node(s) had taint that pod didn't tolerate"
```

Decode a Failing Event

```
kubectl get events \
  --field-selector involvedObject.name=<pod-name> \
  --sort-by=.metadata.creationTimestamp
```

Debug with an Ephemeral Container

```
# attach a debug container (Kubernetes 1.23+)
kubectl debug -it <pod-name> \
  --image=busybox \
  --target=<container-name>
```

Diagnosing with Ephemeral Debug Pods

When a pod has no shell (distroless images), use a debug pod on the same node:

```
# create a privileged debug pod on a specific node
kubectl debug node/<node-name> \
  -it \
  --image=ubuntu \
  -- bash

# inside the debug pod, the node filesystem is at /host
ls /host/etc/kubernetes/manifests/
chroot /host    # enter the node's root filesystem
```

```
# copy a running pod and add a debug container
kubectl debug <pod-name> \
  -it \
  --image=busybox \
  --copy-to=debug-pod \
  --share-processes
```

> Ephemeral containers are the modern replacement for `kubectl exec` when the target container has no debugging tools. They share the pod's namespace and lifecycle.

Cluster Upgrade with kubeadm

Kubernetes releases a new minor version every ~4 months. Always upgrade **one minor version at a time** (e.g., 1.31 → 1.32, never 1.31 → 1.33).

Upgrade Order

```
1. Upgrade the control plane node
   a. Upgrade kubeadm
   b. kubeadm upgrade plan
   c. kubeadm upgrade apply
   d. Upgrade kubelet and kubectl
   e. Restart kubelet

2. Upgrade each worker node
   a. Drain the node
   b. Upgrade kubeadm, kubelet, kubectl
   c. kubeadm upgrade node
   d. Restart kubelet
   e. Uncordon the node
```

> The **control plane must always be upgraded first**. kubelet on worker nodes can be one minor version behind the API server, but never ahead.

Upgrading the Control Plane

Run on the **control plane node**:

```
# 1. update the package repository to the new version
sudo sed -i 's|/v1.34/|/v1.35/|' /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update

# 2. upgrade kubeadm
sudo apt-mark unhold kubeadm
sudo apt-get install -y kubeadm=1.35.0-1.1
sudo apt-mark hold kubeadm

# 3. review the upgrade plan
sudo kubeadm upgrade plan

# 4. apply the upgrade
sudo kubeadm upgrade apply v1.35.0

# 5. drain the control plane (optional for single-node control plane)
kubectl drain cp --ignore-daemonsets

# 6. upgrade kubelet and kubectl
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.35.0-1.1 kubectl=1.35.0-1.1
sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload && sudo systemctl restart kubelet
```

Upgrading Worker Nodes

Run for **each worker node** (drain from control plane, upgrade on worker):

```
# from the control plane — drain the worker
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# SSH into worker-1 and run:
sudo sed -i 's|/v1.34/|/v1.35/|' /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update

sudo apt-mark unhold kubeadm kubelet kubectl
sudo apt-get install -y kubeadm=1.35.0-1.1 kubelet=1.35.0-1.1 kubectl=1.35.0-1.1
sudo apt-mark hold kubeadm kubelet kubectl

sudo kubeadm upgrade node

sudo systemctl daemon-reload && sudo systemctl restart kubelet

# back on the control plane — uncordon the worker
kubectl uncordon worker-1
```

```
# verify the upgraded node
kubectl get nodes
```

Repeat for each remaining worker node.

etcd Backup and Restore

etcd holds **all cluster state**. Regular backups are essential.

Take a Snapshot

```
export ETCDCTL_API=3

sudo etcdctl snapshot save \
  /tmp/etcd-backup-$(date +%Y%m%d).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# verify the snapshot
sudo etcdctl snapshot status \
  /tmp/etcd-backup-$(date +%Y%m%d).db \
  --write-out=table
```

Restore from a Snapshot

```
# stop the API server (remove static pod manifest)
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp

# restore the snapshot to a new data directory
sudo etcdctl snapshot restore \
  /tmp/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored

# update etcd to use the new data directory
sudo sed -i 's|/var/lib/etcd|/var/lib/etcd-restored|' \
  /etc/kubernetes/manifests/etcd.yaml

# restore the API server
sudo mv /tmp/kube-apiserver.yaml \
  /etc/kubernetes/manifests/

# verify cluster is back
kubectl get nodes
```

Cluster Maintenance Operations

```
# cordon a node — prevent new pods from scheduling
kubectl cordon worker-1

# drain a node — evict all pods and cordon
kubectl drain worker-1 \
  --ignore-daemonsets \      # don't evict DaemonSet pods
  --delete-emptydir-data     # evict pods with emptyDir volumes

# uncordon a node — allow scheduling again
kubectl uncordon worker-1

# force delete a stuck pod (lost node)
kubectl delete pod <name> --force --grace-period=0

# remove a node from the cluster
kubectl delete node worker-2
# on worker-2: sudo kubeadm reset
```

Node Lifecycle

```
Healthy → Cordoned → Drained → Maintenance → Uncordoned → Healthy
```

Cluster Design Considerations

Decision	Options	Trade-offs
Control plane HA	1 node vs 3+ nodes	Single point of failure vs complexity
etcd topology	Stacked (on CP nodes) vs External	Simpler vs more resilient
Node size	Many small vs few large	Bin packing efficiency vs blast radius
Upgrade strategy	In-place vs blue/green	Simpler vs zero-downtime
Managed vs self-managed	EKS/GKE/LKE vs kubeadm	Less ops burden vs more control

HA Control Plane with kubeadm

```
# initialize HA control plane with a load balancer VIP
sudo kubeadm init \
  --control-plane-endpoint="<lb-ip>:6443" \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16

# join additional control plane nodes
sudo kubeadm join <lb-ip>:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

📺 HA Control Plane Tutorial Video

Hands-On Labs

Security, Monitoring, Upgrades, and Backup/Restore

Lab 1: RBAC and Access Control

Create a Developer User

```
# generate key and CSR for alice in the dev group
openssl genrsa -out alice.key 2048
openssl req -new -key alice.key -out alice.csr -subj "/CN=alice/O=developers"

# sign with cluster CA
sudo openssl x509 -req -in alice.csr \
  -CA /etc/kubernetes/pki/ca.crt \
  -CAkey /etc/kubernetes/pki/ca.key \
  -CAcreateserial -out alice.crt -days 365
```

Add alice to kubeconfig

```
kubectl config set-credentials alice \
  --client-certificate=alice.crt \
  --client-key=alice.key

kubectl config set-context alice-context \
  --cluster=kubernetes \
  --namespace=default \
  --user=alice
```

Lab 1: RBAC (cont.)

Grant alice read-only access to pods in default namespace

```
kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  -n default

kubectl create rolebinding alice-pod-reader \
  --role=pod-reader \
  --user=alice \
  -n default
```

Test Alice's Permissions

```
# switch to alice's context
kubectl config use-context alice-context

kubectl get pods              # should succeed
kubectl delete pod nginx      # should fail (Forbidden)
kubectl get deployments       # should fail (Forbidden)
kubectl get pods -n kube-system   # should fail (wrong namespace)

# switch back to admin
kubectl config use-context kubernetes-admin@kubernetes
```

Lab 1: RBAC (cont.)

Verify with auth can-i

```
kubectl auth can-i list pods --as=alice
kubectl auth can-i delete pods --as=alice
kubectl auth can-i list pods --as=alice -n kube-system
kubectl auth can-i list nodes --as=alice
```

Create a ServiceAccount for a Workload

```
kubectl create serviceaccount app-reader -n default

kubectl create role deployment-reader \
  --verb=get,list,watch \
  --resource=deployments,replicasets \
  -n default

kubectl create rolebinding app-reader-binding \
  --role=deployment-reader \
  --serviceaccount=default:app-reader \
  -n default

# verify
kubectl auth can-i list deployments \
  --as=system:serviceaccount:default:app-reader
```

Lab 2: Multi-Node Cluster Operations

Drain and Uncordon a Worker Node

```
# check pods on worker-1
kubectl get pods -o wide | grep worker-1

# drain worker-1
kubectl drain worker-1 \
  --ignore-daemonsets \
  --delete-emptydir-data

# verify no non-daemonset pods on worker-1
kubectl get pods -o wide

# perform "maintenance" (nothing to do in lab)
kubectl describe node worker-1 | grep Taints

# uncordon
kubectl uncordon worker-1

# verify pods reschedule
kubectl get pods -o wide -w
```

Simulate a Node Failure

Lab 3: Monitoring Setup

Install kube-prometheus-stack with Helm

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --set grafana.adminPassword=admin123

# watch everything come up
kubectl get pods -n monitoring -w
```

Access Grafana

```
kubectl port-forward svc/monitoring-grafana \
  3000:80 -n monitoring &
```

Open http://localhost:3000, log in with `admin` / `admin123`.

Navigate to **Dashboards → Browse** and explore:

Lab 3: Monitoring (cont.)

Generate Some Load and Watch the Dashboard

```
# deploy a load-generating pod
kubectl run load-test \
  --image=busybox \
  -- sh -c "while true; do wget -q -O- http://web.default.svc.cluster.local; done"

# watch CPU usage spike in Grafana
# Dashboards → Kubernetes / Pods → filter by namespace=default
```

Query Prometheus Directly

```
kubectl port-forward svc/monitoring-kube-prometheus-prometheus \
  9090:9090 -n monitoring &
```

Open http://localhost:9090 and run:

```
# pods with more than 1 restart
kube_pod_container_status_restarts_total > 1

# memory usage by pod
container_memory_working_set_bytes{namespace="default"}
```

Lab 4: Cluster Upgrade

Check Current Version

```
kubectl get nodes
kubeadm version
kubectl version
```

Upgrade the Control Plane

```
# update apt repo to target version
sudo sed -i 's|/v1.34/|/v1.35/|' /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update

# upgrade kubeadm
sudo apt-mark unhold kubeadm
sudo apt-get install -y kubeadm=1.35.0-1.1
sudo apt-mark hold kubeadm

# review the plan
sudo kubeadm upgrade plan

# apply
sudo kubeadm upgrade apply v1.35.0

# upgrade kubelet and kubectl
```

Lab 4: Cluster Upgrade (cont.)

Upgrade a Worker Node

```
# from control plane: drain worker-1
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# SSH into worker-1
sudo sed -i 's|/v1.34/|/v1.35/|' /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-mark unhold kubeadm kubelet kubectl
sudo apt-get install -y kubeadm=1.35.0-1.1 kubelet=1.35.0-1.1 kubectl=1.35.0-1.1
sudo apt-mark hold kubeadm kubelet kubectl
sudo kubeadm upgrade node
sudo systemctl daemon-reload && sudo systemctl restart kubelet

# from control plane: uncordon
kubectl uncordon worker-1

# verify
kubectl get nodes
```

Lab 5: etcd Backup and Restore

Take a Backup

```
export ETCDCTL_API=3

sudo etcdctl snapshot save /tmp/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

sudo etcdctl snapshot status /tmp/etcd-backup.db --write-out=table
```

Simulate Data Loss and Restore

```
# create a resource to prove it exists in etcd
kubectl create namespace before-backup
kubectl create deployment test-deploy --image=nginx -n before-backup

# now "destroy" etcd data (for lab purposes only)
sudo mv /var/lib/etcd /var/lib/etcd-corrupted

# restore from snapshot
sudo etcdctl snapshot restore /tmp/etcd-backup.db \
  --data-dir=/var/lib/etcd
```

Day 4 Recap

What We Covered Today

- ✅ RBAC – Roles, ClusterRoles, RoleBindings, ServiceAccounts
- ✅ SecurityContext – non-root, read-only filesystem, capabilities
- ✅ Pod Security Standards – privileged, baseline, restricted
- ✅ Secrets Encryption at Rest
- ✅ Helm – chart installation, values, upgrades
- ✅ kube-prometheus-stack – Prometheus, Grafana, Alertmanager
- ✅ Troubleshooting – systematic approach, pods, nodes, networking
- ✅ Cluster Upgrades – kubeadm upgrade procedure
- ✅ etcd Backup and Restore

Course Complete

What You Can Now Do

- Build and manage Kubernetes clusters from scratch with kubeadm
- Deploy, scale, and update applications safely
- Configure networking, storage, and ingress
- Secure workloads with RBAC and SecurityContexts
- Monitor cluster health with Prometheus and Grafana
- Systematically troubleshoot any Kubernetes issue
- Upgrade clusters and recover from failures

Next Steps

- **CKA** – Certified Kubernetes Administrator (uses everything from this course)
- **CKAD** – Certified Kubernetes Application Developer
- **CKS** – Certified Kubernetes Security Specialist
- **KubeSkills** – https://kubeskills.com
- **Acing the CKA** – acingthecka.com

Thank Ya